

LAS ESTRUCTURAS DINAMICAS DE DATOS: LOS PUNTEROS (listas y árboles)

CONTENIDO

- 17.1. Punteros
- 17.2. Operaciones con variables punteros: los procedimientos **New** y **Dispose**
- 17.3. El tipo genérico puntero (pointer)
- 17.4. La asignación de memoria en Turbo Pascal
- 17.5. Listas enlazadas
- 17.6. Listas circulares
- 17.7. Árboles
- 17.8. Árbol binario
- 17.9. Puesta a punto de programas

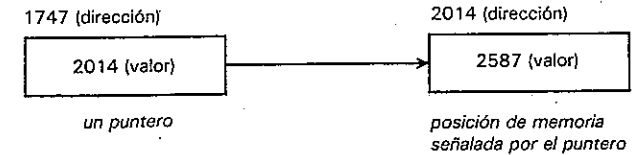
RESUMEN
EJERCICIOS
PROBLEMAS

Las técnicas de programación moderna exigen tratar con estructuras de datos dinámicas que al contrario que las estructuras estáticas ya tratadas se almacenan en zonas de memoria de tamaño variable cuyos valores crecen o decrecen en función del diseño de la estructura. El dato tipo puntero es el elemento base de estructuras dinámicas tales como listas y árboles.

17.1. PUNTEROS

En una computadora cada posición de memoria tiene una dirección y un valor específico almacenado en esa posición. Se han utilizado nombres de variables en lugar de direcciones porque los nombres son más fáciles de recordar. Para almacenar un nuevo valor en memoria se asigna a una variable, y la computadora envía una dirección a la memoria seguida por el valor a almacenar en esa posición.

Pascal proporciona un *tipo especial de variable* denominado **puntero**, que es una variable cuyo valor es una dirección de una posición de memoria.



Al definir un tipo puntero se debe indicar el tipo de valores que se almacenarán en las posiciones designadas por los punteros. La razón es que los diferentes tipos de datos requieren diferentes cantidades de memoria para almacenar sus constantes, una variable puntero puede contener una dirección de una posición de memoria adecuada sólo para un tipo dado. Por esta razón se dice que un puntero *apunta* a una variable particular, es decir, a otra posición de memoria.

Una variable tipo puntero contiene la dirección de la posición de otra variable.

Para declarar una *variable puntero* en Pascal se debe especificar el nombre de la variable y el tipo del valor que se almacenará en la posición de memoria a la que el puntero se refiere.

```
type
  tipo_puntero = ^ tipo dato
```

Una variable de tipo *tipo_puntero* es un *puntero* hacia un dato de tipo *nombre del tipo*. El signo '^' se lee "*apunta hacia*".

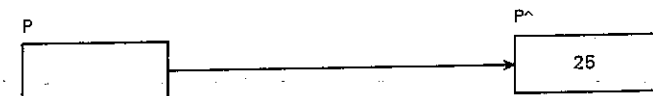
Ejemplos

```
1. type
   Tipo_Puntero = ^ integer;
   var
   P : Tipo_Puntero;
```

```
2. var
   p: ^ integer;
```

```
3. type
   PunteroEntero = ^ integer;
   type
   PunteroCar = ^ char;
   var
   P,Q : PunteroEntero;
   R : PunteroCar;
```

define tipo cuyos valores apuntan a posiciones que contienen enteros
define un tipo cuyos valores apuntan a posiciones que contienen caracteres
p y q apuntan a posiciones que contienen enteros
r apunta a posiciones que contienen caracteres



```

4. type
  Entrada = record
    Nombre : string [20];
    ID      : 1..maxint;
    Coste   : real;
  end
var
  P : ^integer;
  Q : ^char;
  R : ^Entrada;

```

define tres variables punteros P, Q y R. P está limitado a tipos de datos enteros; es decir, P sólo puede contener direcciones de posiciones de memoria adecuada para almacenar enteros. Q apunta a posiciones de memoria apropiadas a caracteres y R se limita a datos de tipo registro Entrada.

```

5. var
  Primero : ^Real;
type
  PunteroInt = ^integer;
var
  Sig : PunteroInt;

```

17.2. OPERACIONES CON VARIABLES PUNTEROS: LOS PROCEDIMIENTOS NEW Y DISPOSE

Los punteros se crean con las declaraciones citadas:

```

type
  PunteroReal = ^Real;
var
  P : PunteroReal;

```

P es una variable puntero de tipo *PunteroReal* que apunta a posiciones que contienen reales.

La posición de memoria designada por el valor de la variable puntero P se representa por P[^]. La Figura 17.1 representa la relación entre P y P[^].

Como P[^] designa una posición de memoria, se puede utilizar como cualquier otra variable Pascal. Se pueden asignar valores a P[^] y utilizar valores de P[^] en expresiones tal como cualquier otra variable. Si P apunta a posiciones que contienen reales, P[^] es una variable real. Así en la Figura 17.1 el valor de P[^] es 50.

P[^] := 1000 el valor de P[^] es 1000
 3 * P[^] + 500 toma el valor 3500

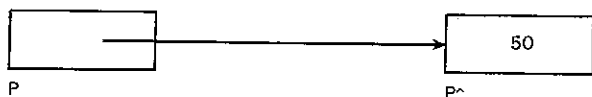


Figura 17.1. Diferencias entre P y P[^].

Sin embargo, estas operaciones no se pueden realizar directamente tras la declaración, debido a que el objeto o dirección apuntada P[^] no tiene existencia. Antes de que un programa utilice un puntero, se requiere primero espacio para el tipo de datos objeto de la dirección del puntero. En otras palabras, un programa debe inicializar sus punteros —su declaración no basta—; para inicializar un puntero se debe utilizar el procedimiento New.

17.2.1. New

El procedimiento New crea una nueva variable dinámica y establece que una variable puntero apunte a ella.

Formato

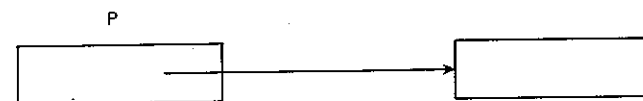
New (P)

p variable puntero

La nueva variable creada es la ya conocida con el símbolo P[^]. La llamada a New exige que exista suficiente espacio de memoria libre en el *montículo* —la pila de variables dinámicas— (*heap*) para asignar a la nueva variable. En caso contrario se producirá un error en tiempo de ejecución.

New(P)

Llama al procedimiento New que asigna almacenamiento para un valor de tipo Real y sitúa la dirección de esta posición de memoria en la variable puntero P.



La sentencia de asignación

P[^] := 25.0

almacena el valor real en la posición de memoria P.



Como P[^] se considera una variable, podrá ser visualizado su valor

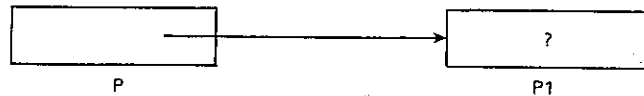
Write (P[^]: 10: 2); visualiza el valor 25.0

Atención

P := 25.0 asignación no válida
 Write (P : 10 : 2); sentencia no válida

Funcionamiento de New (P)

1. Se asigna espacio de memoria conveniente para almacenar tipos de datos correspondientes.
2. La dirección de esta posición de memoria se almacena en P; se crea una *variable dinámica* P[^].



Ejemplo 17.1

Realizar el seguimiento del segmento de programa.

```

var
1.  Ptr1 : ^char;
2.  Ptr2 : ^integer;
begin
3.  New (Ptr1);
4.  Ptr1^ := 'B';
5.  New (Ptr2);
6.  Ptr2^ := 86;

```

Paso	Gráfico de memoria	Significa
1	 Ptr1	Se declara Ptr1 como una variable estática
2	 Ptr1 Ptr2	Se declara Ptr2

Paso	Gráfico de memoria	Significa
3	 Ptr1 Ptr2	Asigna espacio de memoria para Ptr1 [^] y almacena su dirección en Ptr1 (se crea una nueva posición de memoria)
4	 Ptr1 Ptr2	Asignación del valor B a la variable dinámica Ptr1 [^]
5	 Ptr1 Ptr2	Asigna espacio de memoria para Ptr2 [^] y almacena su dirección en Ptr2
6	 Ptr1 Ptr2	Asignación de 86 a la variable dinámica Ptr2 [^]

Ejemplo 17.2

Realice el lector el seguimiento del siguiente programa y deduzca su salida.

```

program Puntero;
type
  Ptring = ^integer;
var
  I, J : Ptring;
  N : integer;
begin
  New (I);
  New (J);

```

```

N := 5;
I^ := N;
WriteLn (I^);
J := I;
J^ := -7;
WriteLn (I^);
end.

```

17.2.2. Dispose

El procedimiento **dispose** libera la posición de memoria ocupada por una variable dinámica.

Formato

dispose (p)

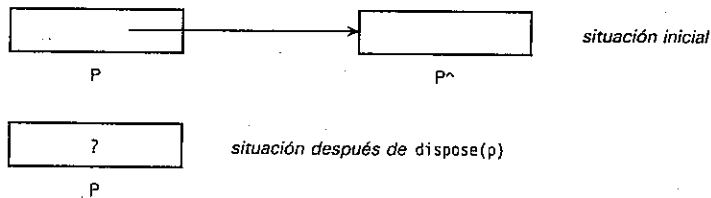
p variable puntero

Dispose destruye la variable referenciada por *P* y devuelve su zona de memoria a la pila dinámica (*heap*). Después de una llamada a *dispose*, el valor de *P* se vuelve indefinido y cualquier intento de referenciar a *P^* producirá un error.

En resumen, *Dispose* produce las siguientes acciones:

1. La posición de memoria cuya dirección se almacena en *P* se libera de la memoria montículo (*heap*). En otras palabras, *P^* deja de existir.
2. *P* se vuelve indefinido.

La representación gráfica es:



Ejemplo 17.3

```

1. var
   P, Q : ^integer;
begin
2.   New (P);

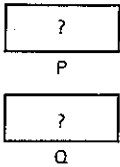
```

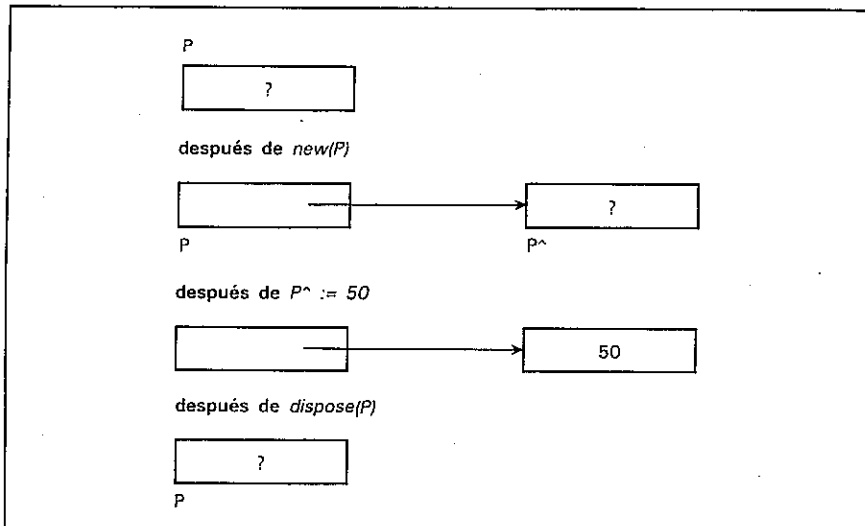
```

3.   P^ := 8;
4.   Q := P;
5.   Q^ := 5;
6.   Dispose (P)
end

```

Paso	Gráfico de memoria	Significa
1		Declaración de las variables puntero
2		Asignación de memoria a P^
3		Asignación a P^
4		El contenido de P se asigna a Q de modo que P^ y Q^ comparten la misma posición de memoria
5		Asignación a Q^ y P^

Paso	Gráfico de memoria	Significa
6		Como P y Q apuntan a la misma posición, <i>dispose</i> libera la memoria de P y Q.

Funcionamiento de *new* y *dispose*

17.2.3. Inicialización y asignación de punteros

Antes de intentar referirse a valores de variables es preciso, como siempre, inicializar las variables. Los pasos a dar se muestran en las siguientes sentencias:

```
var
  P : ^integer;
  .
  .
  .
New (P);
  .
  .
  .
P^ := 17;
```

En un puntero (P) se deben considerar dos valores:

- *puntero* (P) Su valor es la posición (dirección) de una variable *integer*.
- $P^$ El valor de la variable *integer* propiamente dicho.



El procedimiento *new* inicializa el puntero con la dirección de una variable *integer*. La variable *integer*, $P^$, a la que P apunta debe también ser inicializada ($P^ := 17$).

Recuerde

Write ($P^$) *sentencia válida*
 Write (P) *sentencia no válida (error)*

Asignación

Las variables dinámicas pueden tener más de un puntero que esté apuntando a ellas. También los punteros pueden ser modificados de modo que apunten a variables dinámi-

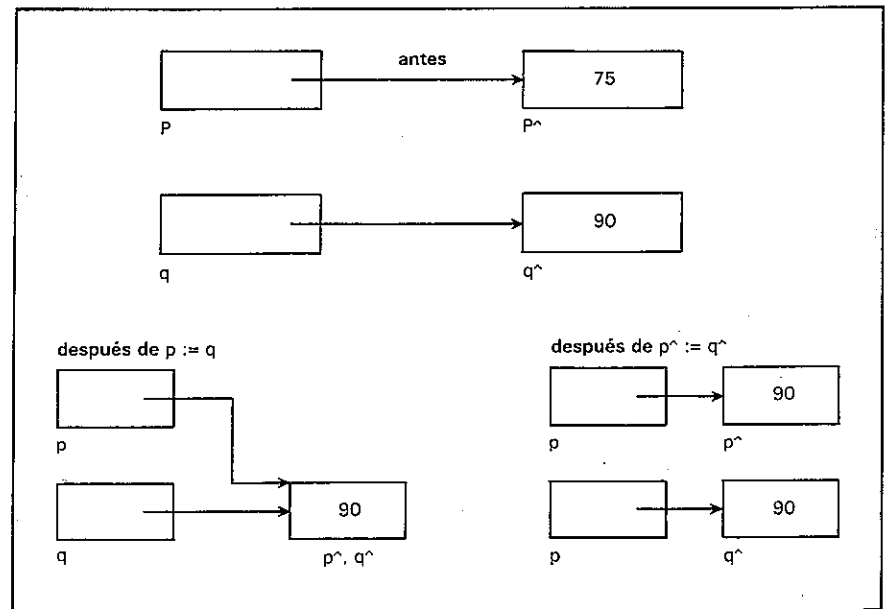


Figura 17.2. Asignaciones entre punteros.

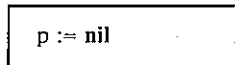
cas diferentes en momentos diferentes. Estos cambios se pueden realizar con el operador asignación ($:=$). Estudiemos las asignaciones posibles.



- (1) Los punteros p y q apuntan a la misma posición; por consiguiente, $p^$ y $q^$ designan la misma posición y tienen el mismo valor.
- (2) El valor de $q^$ se asigna a $p^$; después de la asignación, $p^$ y $q^$ tienen el mismo valor; sin embargo, $p^$ y $q^$ permanecen en distintas posiciones de memoria.

17.2.4. Constante nil

Pascal proporciona una constante predefinida, **nil** (nulo). La constante **nil** se utiliza para dar un valor a una variable puntero que no apunta a ninguna posición, nil puede ser asignada a un puntero de cualquier tipo.



Como **nil** no apunta a ninguna posición de memoria, una referencia a $p^$ es ilegal si el valor de p es **nil**.

```
p := nil;
p^ := 5
```

es ilegal ya que se instruye a la computadora a almacenar 5 en la posición apuntada por p, pero debido a la primera sentencia (p:=nil), p no apunta a ninguna dirección

Ejemplo 17.4

El segmento de programa

```
var
  p : ^char;
begin
  p := nil;
  if p = nil
  then
    WriteLn ('el puntero P no apunta a nada')
  else
    WriteLn ('P se queda indefinido apunta a una variable
      caracter');
```

proporciona la salida

el puntero p no apunta a nada

17.2.5. Variables de puntero con registros

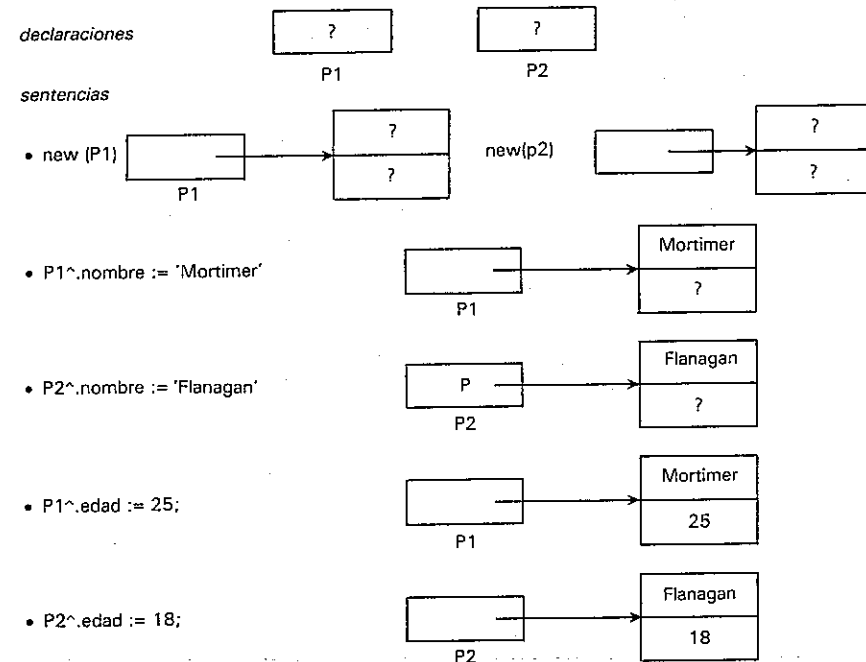
Las variables de puntero pueden contener la dirección de tipos de datos simples o datos de tipo compuesto. En las estructuras dinámicas de datos es frecuente el uso de registros como elementos. Se suelen conocer con el nombre de *nodos*, aunque en realidad el término se extiende tanto a datos simples como compuestos. Por su importancia, y aunque la manipulación es idéntica a la ya estudiada, consideraremos un ejemplo con variables dinámicas tipo registro.

Ejemplo

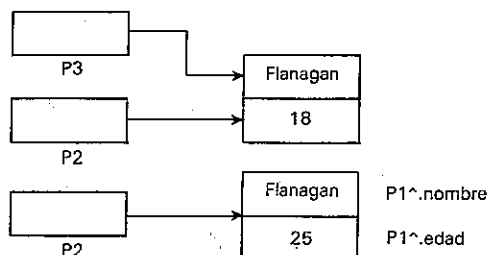
```
type
  Estudiante = record
    Nombre : string [20];
    Edad   : integer;
  end;
  PunteroEst = ^Estudiante;
var
  P1, P2, P3 : PunteroEst;
  Write (P1^.nombre, P2^.nombre, P3^.nombre);
```

puede ser sustituido por
var P1, P2 : Estudiante

Algunas operaciones típicas son:



• P3 := P2;



• write (P1^.nombre, P2^.nombre, P3^.nombre)
visualiza las palabras *nombres* de los registros apuntados por P1, P2 y P3.

Mortimer Flanagan Flanagan

Ejemplo 17.5

¿Cuál es la salida de este programa?

```

program Test;
type
  Estudiante = record
    Letra : char;
    Edad : integer;
  end;
  PuntEstu = ^Estudiante;
var
  P1, P2 : PuntEstu;
begin
  New (P1);
  P1^.Edad := 1;
  P1^.Letra := 'A';
  WriteLn (P1^.Edad, P1^.Letra);
  New (P2);
  P2^.Edad := 2;
  P2^.Letra := 'B';
  WriteLn (P2^.Edad, P2^.Letra);
  P1 := P2;
  P2^.Edad := 3;
  P2^.Letra := 'C';
  WriteLn (P1^.Edad, P1^.Letra, P2^.Edad, P2^.Letra)
end.
  
```

Ejecución

1A
2B
3C 3C

17.2.6. Comparación de punteros

Los punteros pueden ser comparados *sólo* en expresiones de igualdad. Así en el caso de

```

var
  PunteroA, PunteroB : ^integer;
  
```

Las comparaciones:

```

if (PunteroA = PunteroB)
  then <sentencia>;
if (PunteroA <> PunteroB)
  then <sentencia>;
  
```

No se pueden comparar la ordenación de punteros con los operadores de relación <, >, <= y >=.

No existe ningún problema para comparar los objetos direccionados por los punteros. La línea siguiente es variable.

```

if (PunteroA^ <= PunteroB^ )
  then b <sentencia>
  
```

17.3. EL TIPO GENERICO PUNTERO (POINTER)

Turbo Pascal permite un tipo especial de definición de puntero: “*genérico*” o “*no tipeado*”. Difiere del puntero estándar en que no tiene un tipo base, no está definido como un puntero hacia algún tipo, sino simplemente como una variable de tipo **pointer**.

Ejemplo

```

var
  enlace : pointer;
  ...
  enlace := lis; {la variable lis figura en declaraciones
                  precedentes}
  ...
  
```

La dirección contenida en la variable *lis* está asignada a la variable *enlace*.

Los punteros genéricos están especialmente concebidos para la programación de bajo nivel, para la cual Turbo Pascal ofrece buenas posibilidades.

17.4. LA ASIGNACION DE MEMORIA EN TURBO PASCAL

Turbo Pascal divide la memoria de su computadora en cuatro partes: el segmento de código, el segmento de datos, el segmento pila (*stack*) y el segmento *montículo* o *almace-*

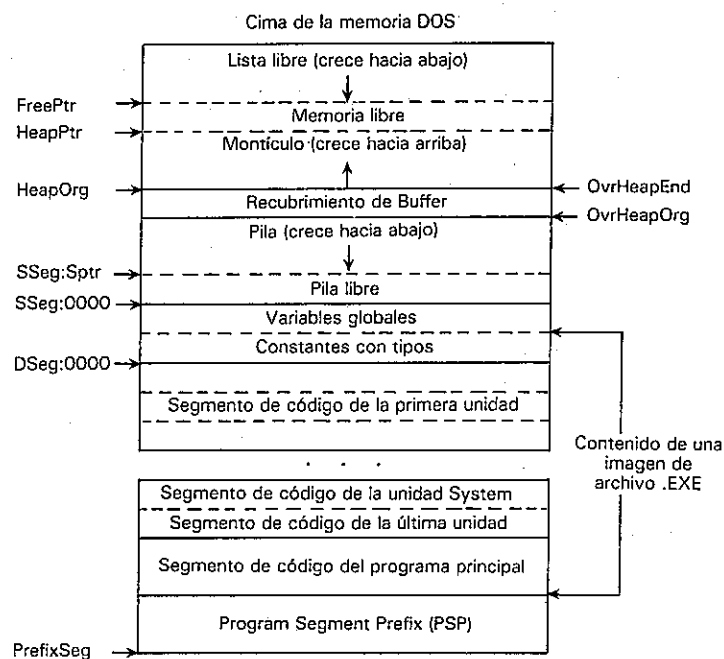


Figura 17.3. Mapa de memoria Turbo Pascal 5.X

namiento dinámico (heap). Técnicamente la pila y el montículo no están totalmente separados, pero funcionan como entidades separadas.

El segmento de datos está claramente dedicado al almacenamiento de datos, pero en los otros tres segmentos también pueden almacenarse datos. La Figura 17.3 muestra el mapa de memoria simplificado de Turbo Pascal 5. Cada módulo (que incluye el programa principal y cada unidad) tiene su propio segmento de código. El programa principal ocupa el primer segmento de unidades (en orden inverso de como están listadas en la cláusula *uses*) y el último segmento de código está ocupado por la librería en tiempo de ejecución.

El tamaño de un segmento de código no puede exceder de 64k, pero el tamaño total del código está limitado sólo por la memoria disponible. El segmento de datos contiene todas las constantes con tipo seguidas por todas las variables globales. El tamaño del segmento de la pila no puede exceder de 64k (el tamaño por defecto es 16k, que se pueden modificar con la directiva *\$M*).

El *buffer* o memoria intermedia de recubrimiento (*overlay*) se utiliza por la unidad estándar *Overlay* para almacenar código recubierto. Si el programa no tiene *recubrimiento*, el tamaño de la memoria intermedia del recubrimiento es cero.

La Figura 17.4 (modificación de la 17.3) muestra cómo queda la memoria cuando un programa arranca, y en ella se observa que todas las variables locales se almacenan en la

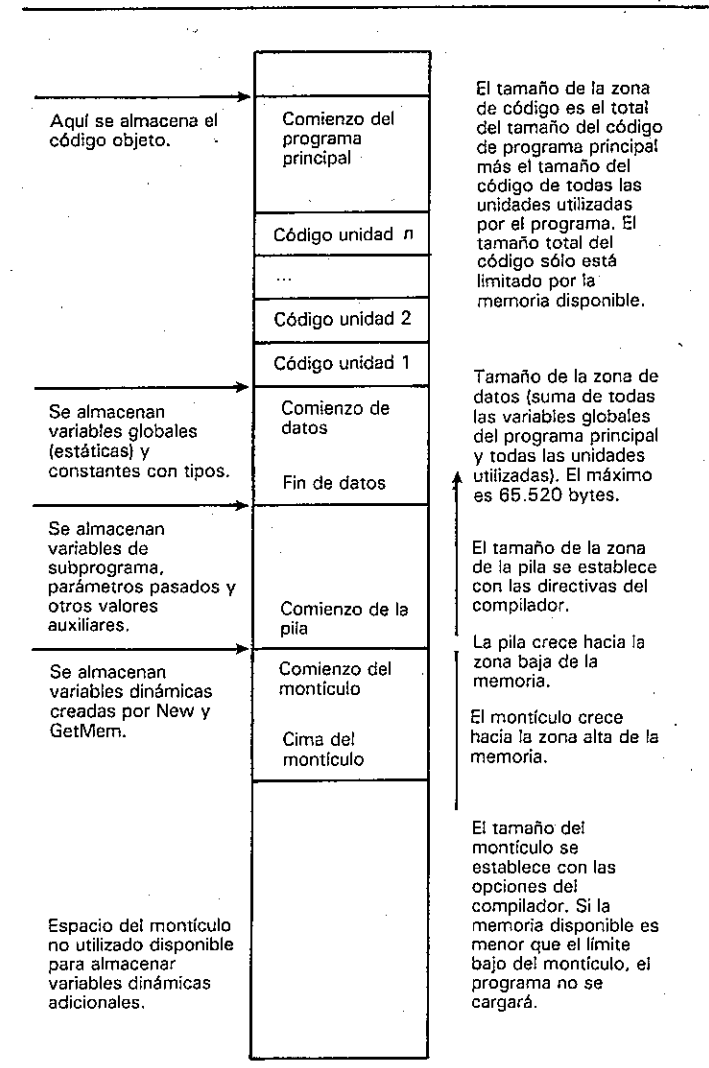


Figura 17.4. Mapa práctico de memoria de Turbo Pascal 5.0.

pila (*stack*) y las variables globales (también llamadas estáticas) se almacenan en el segmento de datos. El código y el segmento de datos están localizados en la parte baja de la memoria y la pila (*stack*) y el almacenamiento dinámico o montículo (*heap*) ocupan la zona alta de la memoria.

El diagrama de la Figura 17.4 muestra también que la pila crece hacia abajo en la memoria y el montículo crece hacia arriba en la memoria. Aunque la pila y el montículo compartan la misma zona de la memoria, ellas nunca deben solaparse (recubrirse).

La mayoría de las variables que se declaran en Turbo Pascal son estáticas, su tamaño se fija en tiempo de compilación y no pueden variar. Por el contrario, el montículo almacena variables dinámicas.

17.4.1. El montículo (heap) y los punteros

El **montículo** o **heap** (*pila de variables dinámicas* o *almacenamiento dinámico*) almacena *variables dinámicas*, esto es, las variables asignadas a través de los procedimientos estándar *New* y *GetMem*. El montículo puede crecer o disminuir en el segmento correspondiente, ya que utiliza tipos de datos dinámicos: los *punteros*, que pueden crear o liberar variables dinámicas mientras el programa se está ejecutando. El resumen, las variables puntero pueden utilizar y reutilizar la memoria montículo.

El tamaño real del montículo depende de los valores mínimos y máximos que pueden fijarse con la directiva del compilador *\$M*. El tamaño mínimo es de 0 bytes, y el máximo por defecto es de 640Kb; esto significa que por defecto el montículo ocupará toda la memoria restante (640Kb viene definida por la máxima memoria direccionable por el DOS, aunque los procesadores 8086/88 tienen dieciséis segmentos que por un valor de 64k de RAM resultaría 1.048.560 bytes = 1 *MegaByte*).

El límite inferior del montículo se almacena en la variable *HeapOrg*, y el límite o cota superior (límite inferior de la memoria libre) se almacena en la variable *HeapPtr*. Cada vez que una variable dinámica se asigna en el montículo (vía *New* o *GetMem*), el gestor de la pila mueve *HeapPtr* hacia arriba el tamaño de la variable.

El uso de las variables puntero en el montículo ofrece dos ventajas principales: Primero se amplía la cantidad total del espacio de datos disponibles en un programa; el segmento de datos está limitado a 64k, pero el montículo, como ya se ha citado, está limitado sólo por la cantidad de RAM en su computadora. La segunda ventaja es permitir que su programa se ejecute con menos memoria. Por ejemplo, un programa puede tener dos estructuras de datos muy grandes, pero sólo una de ellas se utiliza en cada momento. Si estas estructuras de datos se declaran globalmente, ellas residen en el segmento de datos y ocupan memoria en todo momento. Sin embargo, si estas estructuras de datos se definen como punteros, se pueden poner en el montículo y quitarse cuando no se necesiten, reduciendo, por consiguiente, los requisitos de memoria.

17.4.2. Métodos de liberación de memoria

Los subprogramas que gestionan el montículo o almacenamiento dinámico son:

Asignación dinámica de memoria

New
Mark
GetMem

Dispose
Release

FreeMem

Espacio ocupado en memoria

Max Avail
Min Avail

17.4.3. New y Dispose

Cuando **Dispose** libera o destruye una variable dinámica, puede dejar un agujero en el montículo. Si hace uso frecuente de **New** y **Dispose**, puede llegar a producirse una acumulación de espacio de memoria inservible "basura" (*garbage*). Algunos sistemas Pascal proporcionan rutinas específicas para comprimir esa información inservible (*garbage collection*), Turbo Pascal no las soporta, pero sí utiliza un sistema de gestión de la memoria montículo muy sofisticada que minimiza la pérdida de esos espacios inservibles. Los procedimientos **Mark** y **Release** proporcionan un método alternativo de gestión del montículo que eliminan la necesidad de la operación "eliminar basura". Sin embargo, hay un inconveniente: **Dispose** no es compatible con **Mark** y **Release**.

17.4.4. Mark y Release

Mark y **Release** son una alternativa a utilizar **New** y **Dispose** para asignar memoria dinámicamente.

El procedimiento **Mark** registra la dirección de la parte superior del montículo en una variable puntero. El procedimiento **Release** devuelve el montículo a un estado dado (restaura la dirección).

Mark (varpunt)

Release (varpunt)

varpunt variable puntero de cualquier tipo puntero

Nota

Recuerde que **Dispose** y **Release** son métodos incompatibles de recuperación de memoria. Puede elegir utilizar uno u otro, nunca utilizar ambos en el mismo programa.

17.4.5. GetMem y FreeMem

Un tercer método de asignar memoria dinámica es **GetMem** y **FreeMem**. Se asemejan a **New** y **Dispose**, en que asignan o liberan memoria, pero **GetMem** y **FreeMem** pueden especificar cuánta memoria se desea asignar con independencia del tipo de variable que está utilizando. Se pueden asignar bloques de memoria en la pila de una unidad de datos cuyo tamaño no se conoce en tiempo de compilación.

GetMem crea una nueva variable dinámica del tamaño especificado y pone la dirección del bloque en una variable puntero.

Formato

GetMem (varpunt, tamaño)

varpunt variable puntero de cualquier tipo puntero
tamaño expresión de tipo word

FreeMem libera una variable dinámica de un tamaño dado.

Formato

FreeMem (varpunt, tamaño)

Las variables asignadas con GetMem se liberan con FreeMem.

Ejemplo

```
GetMem (i, 100);
i := x+y;
WriteLn (i);
FreeMem (i, 100);
```

Nota

El número de bytes especificado en FreeMem debe concordar con el especificado en GetMem. No se debe utilizar Dispose en lugar de FreeMem.

17.4.6. MemAvail y MaxAvail

El conocimiento de cuánta memoria está realmente disponible para variables dinámicas puede ser crítico. Turbo Pascal lleva un registro de cuánta memoria queda en el montículo, de modo que si se solicita más de la disponible, se generará un error en tiempo de ejecución. La solución es no crear una variable dinámica que sea mayor que la memoria disponible en el montículo. Turbo Pascal proporciona dos funciones para medir la memoria disponible: MemAvail y MaxAvail.

MaxAvail devuelve el tamaño del bloque libre contiguo más grande del montículo, correspondiente al tamaño de la variable dinámica más grande que se puede asignar a la vez. (El valor devuelto es un entero largo.)

Formato

MaxAvail

MemAvail devuelve un valor (del tipo entero largo) que indica, en bytes, la cantidad total de memoria disponible para variables dinámicas.

Formato

MemAvail

Ejemplos

```
begin
  WriteLn (MemAvail, 'bytes disponibles');
  WriteLn ('el bloque libre mas grande es: ', MaxAvail, 'bytes');
end
```

17.5. LISTAS ENLAZADAS

Cuando se procesan conjuntos de datos cuyo espacio de almacenamiento no se puede predecir a priori (en tiempo de compilación) y además la actividad de los mismos (inserciones y borrados) es frecuente, las estructuras de datos estáticas (los arrays) no son adecuadas para su implementación. Las razones son varias:

1. Los arrays deben ser declarados en tamaño en el programa fuente, de modo que si se elige uno mayor que el necesario, entonces se malgasta espacio de memoria (por ejemplo, dimensionar el array a 100 elementos y utilizar sólo 20), y si se hace el array pequeño, podría no ejecutarse el programa.
2. La operación de añadir datos al final de la lista (el array) es un proceso rápido; sin embargo, las inserciones y eliminaciones en el interior del array son lentas y complejas, ya que puede ser necesario desplazar cada elemento del array para hacer espacio al nuevo elemento, o bien cerrar el espacio dejado por una eliminación.

Si a esta dificultad se añaden los casos en que las operaciones anteriores sean frecuentes, se puede deducir que en estos casos las estructuras más idóneas son las estructuras dinámicas de datos. Se clasifican en los siguientes tipos:

listas { listas
pilas
colas

árboles

grafos

Las listas enlazadas son una secuencia de *nodos* que se encuentran enlazados cada uno con el siguiente mediante un *enlace* o *puntero*. Cada elemento (*nodo*) de una lista enlazada debe tener dos campos: un campo (*info*) que contiene el valor de ese elemento y un campo (*enlace* o *puntero*) que indica la posición del siguiente elemento.

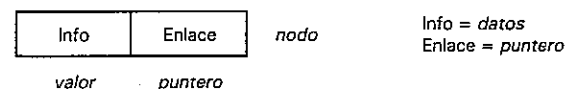


Figura 17.5. Nodo de una lista enlazada